

Built-in kernel self tests



Kernel memory issue debugging

- The same kind of memory issues that can happen in user space can be triggered while writing kernel code
  - Out of bounds accesses
  - Use-after-free errors (dereferencing a pointer after `kfree()`)
  - Out of memory due to missing `kfree()`
- Various tools are present in the kernel to catch these issues
  - *KASAN* to find use-after-free and out-of-bound memory accesses
  - *KFENCE* to find use-after-free and out-of-bound in production systems
  - *Kmemleak* to find memory leak due to missing free of memory



KASAN

- Kernel Address Space Sanitizer
- Allows to find use-after-free and out-of-bounds memory accesses
- Uses GCC to instrument the kernel at compile-time
- Supported by almost all architectures (ARM, ARM64, PowerPC, RISC-V, S390, Xtensa and X86)
- Needs to be enabled at kernel configuration with CONFIG_KASAN
- Can then be enabled for files by modifying Makefile
  - `KASAN_SANITIZE_file.o := y` for a specific file
  - `KASAN_SANITIZE := y` for all files in the Makefile folder



Kmemleak

- Kmemleak allows to find memory leaks for dynamically allocated objects with `kmalloc()`
  - Works by scanning the memory to detect if allocated address are not referenced anymore anywhere (large overhead).
- Once enabled with CONFIG_DEBUG_KMEMLEAK, kmemleak control files will be visible in *debugfs*
- Memory leaks is scanned every 10 minutes
  - can be disabled via CONFIG_DEBUG_KMEMLEAK_AUTO_SCAN
- An immediate scan can be triggered using
  - `# echo scan > /sys/kernel/debug/kmemleak`
- Results are displayed in debugfs
  - `# cat /sys/kernel/debug/kmemleak`
- See dev-tools/kmemleak for more information



Kmemleak report

```
# cat /sys/kernel/debug/kmemleak
unreferenced object 0x82d43100 (size 64):
  comm "insmod", pid 140, jiffies 4294943424 (age 270.420s)
  hex dump (first 32 bytes):
    b4 bb e1 8f c8 a4 e1 8f 8c ce e1 8f 88 c6 e1 8f  ................
    10 a5 e1 8f 18 e2 e1 8f ac c6 e1 8f 0c c1 e1 8f  ................
  backtrace:
    [<c31f5b59>] slab_post_alloc_hook+0xa8/0x1b8
    [<c8200adb>] kmem_cache_alloc_trace+0xb8/0x104
    [<1836406b>] 0x7f005038
    [<89fff56d>] do_one_initcall+0x80/0x1a8
    [<31d908e3>] do_init_module+0x50/0x210
    [<2658dd55>] load_module+0x208c/0x211c
    [<e1d48f15>] sys_finit_module+0xe4/0xf4
    [<1de12529>] ret_fast_syscall+0x0/0x54
    [<7ee81f34>] 0x7eca8c80
```



UBSAN

- UBSAN is a runtime checker for code with undefined behavior
  - Shifting with a value larger than the type
  - Overflow of integers (signed and unsigned)
  - Misaligned pointer access
  - Out of bound access to static arrays
  - https://clang.llvm.org/docs/UndefinedBehaviorSanitizer.html
- It uses compile-time instrumentation to insert checks that will be executed at runtime
- Must be enabled using CONFIG_UBSAN=y
- Then, can be enabled for specific files by modifying Makefile
  - `UBSAN_SANITIZE_file.o := y` for a specific file
  - `UBSAN_SANITIZE := y` for all files in the Makefile folder



UBSAN: report example

- Report for an undefined behavior due to a shift with a value > 32.

```
UBSAN: Undefined behaviour in mm/page_alloc.c:3117:19
shift exponent 51 is too large for 32-bit type 'int'
CPU: 0 PID: 6520 Comm: syz-executor1 Not tainted 4.19.0-rc2 #1
Hardware name: QEMU Standard PC (i440FX + PIIX, 1996), BIOS Bochs 01/01/2011
Call Trace:
__dump_stack lib/dump_stack.c:77 [inline]
dump_stack+0xd2/0x148 lib/dump_stack.c:113
ubsan_epilogue+0x12/0x94 lib/ubsan.c:159
__ubsan_handle_shift_out_of_bounds+0x2b6/0x30b lib/ubsan.c:425
...
RIP: 0033:0x4497b9
Code: e8 8c 9f 02 00 48 83 c4 18 c3 0f 1f 80 00 00 00 00 48 89 f8 48
89 f7 48 89 d6 48 89 ca 4d 89 c2 4d 89 c8 4c 8b 4c 24 08 0f 05 <48> 3d
01 f0 ff ff 0f 83 9b 6b fc ff c3 66 2e 0f 1f 84 00 00 00 00
RSP: 002b:00007fb5ef0e2c68 EFLAGS: 00000246 ORIG_RAX: 0000000000000010
RAX: ffffffffffffffda RBX: 00007fb5ef0e36cc RCX: 00000000004497b9
RDX: 0000000020000040 RSI: 0000000000000258 RDI: 0000000000000014
RBP: 000000000071bea0 R08: 0000000000000000 R09: 0000000000000000
R10: 0000000000000000 R11: 0000000000000246 R12: 00000000ffffffff R13: 0000000000005490 R14: 00000000006ed530
R15: 00007fb5ef0e3700
```



Debugging locking

- Lock debugging: prove locking correctness
  - CONFIG_PROVE_LOCKING
  - Adds instrumentation to kernel locking code
  - Detect violations of locking rules during system life, such as:
    - Locks acquired in different order (keeps track of locking sequences and compares them).
    - Spinlocks acquired in interrupt handlers and also in process context when interrupts are enabled.
  - Not suitable for production systems but acceptable overhead in development.
  - See locking/lockdep-design for details
- CONFIG_DEBUG_ATOMIC_SLEEP allows to detect code that incorrectly sleeps in atomic section (while holding lock typically).
  - Warning displayed in `dmesg` in case of such violation.



Concurrency issues

- Kernel Concurrency SANitizer framework
- CONFIG_KCSAN, introduced in Linux 5.8.
- Dynamic race detector relying on compile time instrumentation.
- Can find concurrency issues (mainly data races) in your system.
- See dev-tools/kcsan and https://lwn.net/Articles/816850/ for details.

Practical lab - Kernel debugging



Debugging kernel programming mistakes with integrated frameworks

- Debug locking issues using lockdep
- Spot function calls in invalid context
- Use kmemleak to detect memory leaks on the system



The Magic SysRq



The Magic SysRq

Functionality provided by serial drivers

- Allows to run multiple debug/rescue commands even when the kernel seems to be in deep trouble
  - On embedded: in the console, send a break character (Picocom: press `[Ctrl] + a` followed by `[Ctrl] + \`), then press `<character>`
  - By echoing `<character>` in `/proc/sysrq-trigger`
- Example commands:
  - `h`: show available commands
  - `s`: sync all mounted filesystems
  - `b`: reboot the system
  - `w`: shows the kernel stack of all sleeping processes
  - `t`: shows the kernel stack of all running processes
  - `g`: enter kgdb mode
  - `z`: flush trace buffer
  - `c`: triggers a crash (kernel panic)
  - You can even register your own!
- Detailed in admin-guide/sysrq



KGDB



kgdb - A kernel debugger

- CONFIG_KGDB in *Kernel hacking*.
- The execution of the kernel is fully controlled by `gdb` from another machine, connected through a serial line.
- Can do almost everything, including inserting breakpoints in interrupt handlers.
- Feature supported for the most popular CPU architectures
- CONFIG_GDB_SCRIPTS allows to build GDB python scripts that are provided by the kernel.
  - See process/debugging/kgdb for more information



kgdb kernel config

- CONFIG_DEBUG_KERNEL=y to make KGDB support visible
- CONFIG_KGDB=y to enable KGDB support
- CONFIG_DEBUG_INFO=y to compile the kernel with debug info (`-g`)
- CONFIG_FRAME_POINTER=y to have more reliable stacktraces
- CONFIG_KGDB_SERIAL_CONSOLE=y to enable KGDB support over serial
- CONFIG_GDB_SCRIPTS=y to enable kernel GDB python scripts
- CONFIG_RANDOMIZE_BASE=n to disable KASLR
- CONFIG_WATCHDOG=n to disable watchdog
- CONFIG_MAGIC_SYSRQ=y to enable Magic SysReq support
- CONFIG_STRICT_KERNEL_RWX=n to disable memory protection on code section, thus allowing to put breakpoints



kgdb pitfalls

- KASLR should be disabled to avoid confusing gdb with randomized kernel addresses
  - Disable *kaslr mode using* `nokaslr` *command line parameter if enabled in your kernel.*
- Disable the platform watchdog to avoid rebooting while debugging.
  - When interrupted by KGDB, all interrupts are disabled thus, the watchdog is not serviced.
  - Sometimes, watchdog is enabled by upper boot levels. Make sure to disable the watchdog there too.
- Can not interrupt kernel execution from gdb using `interrupt` command or `Ctrl + C`.
- Not possible to break everywhere (see CONFIG_KGDB_HONOUR_BLOCKLIST).
- Need a console driver with polling support.
- Some architecture lacks functionalities (No watchpoints on arm32 for instance) and some instabilities might happen!



Using kgdb (1/2)

- Details available in the kernel documentation: process/debugging/kgdb
- You must include a kgdb I/O driver. One of them is `kgdb` over serial console (`kgdboc`: `kgdb` over console, enabled by CONFIG_KGDB_SERIAL_CONSOLE)
- Configure `kgdboc` at boot time by passing to the kernel:
  - `kgdboc=<tty-device>,<bauds>`.
  - For example: `kgdboc=ttyS0,115200`
- Or at runtime using sysfs:
  - `echo ttyS0 > /sys/module/kgdboc/parameters/kgdboc`
  - If the console does not have polling support, this command will yield an error.



Using kgdb (2/2)

- Then also pass `kgdbwait` to the kernel: it makes `kgdb` wait for a debugger connection.
- Boot your kernel, and when the console is initialized, interrupt the kernel with a break character and then `g` in the serial console (see our *Magic SysRq* explanations).
- On your workstation, start `gdb` as follows:
  - `arm-linux-gdb ./vmlinux`
  - `(gdb) set serial baud 115200`
  - `(gdb) target remote /dev/ttyS0`
- Once connected, you can debug a kernel the way you would debug an application program.
- On GDB side, the first threads represent the CPU context (ShadowCPU<x>), then all the other threads represents a task.



Kernel *GDB scripts*

- CONFIG_GDB_SCRIPTS allows to build a set of python script which ease the kernel debugging by adding new commands and functions.
- When using `gdb vmlinux`, the scripts present in vmlinux-gdb.py file at the root of build dir will be loaded automatically.
  - `lx-symbols`: (Re)load symbols for vmlinux and modules
  - `lx-dmesg`: display kernel dmesg
  - `lx-lsmod`: display loaded modules
  - `lx-device-bus|class|tree`: display device bus, classes and tree
  - `lx-ps`: `ps` like view of tasks
  - `$lx_current()` contains the current `task_struct` item `$lx_per_cpu(var, cpu)` returns a per-cpu variable
  - `apropos lx` To display all available functions.
- dev-tools/gdb-kernel-debugging



KDB

- CONFIG_KGDB_KDB includes a kgdb frontend name “KDB”
- This frontend exposes a debug prompt on the serial console which allows debugging the kernel without the need for an external gdb.
- KDB can be entered using the same mechanism used for entering kgdb mode.
- *KDB* and *KGDB* can coexist and be used at the same time.
  - Use the `kgdb` command in KDB to enter kgdb mode.
  - Send a maintenance packet from gdb using `maintenance packet 3` to switch from kgdb to KDB mode.



KDB commands

- KDB does not consume gdb commands but a set of dedicated KDB commands:
  - `go`: Continue execution
  - `bt`: Display backtrace
  - `env`: Show environment variables
  - `ps`: List all tasks
  - `pid`: Switch to another task
  - `md/mm`: Read/write memory
  - `lsmod`: List loaded modules
- To check all available commands, you can refer to the `help` command output, or check maintab in kernel source code

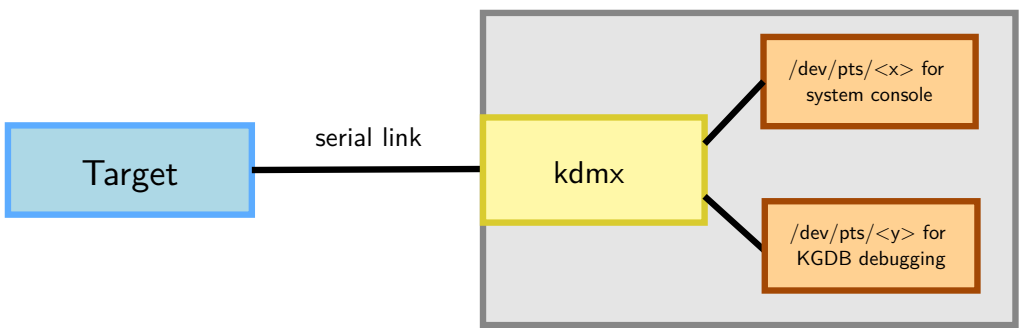


kdmx

- When the system has only a single serial port, it is not possible to use both KGDB and the serial line as an output terminal since only one program can access that port.
- Fortunately, the *kdmx* tool allows to use both KGDB and serial output by splitting GDB messages and standard console from a single port to 2 slave pty (`/dev/pts/x`)
- https://git.kernel.org/pub/scm/utils/kernel/kgdb/agent-proxy.git
  - Located in the subdirectory `kdmx`

```
$ kdmx -n -d -p/dev/ttyACM0 -b115200
serial port: /dev/ttyACM0
Initalizing the serial port to 115200 8n1
/dev/pts/6 is slave pty for terminal emulator
/dev/pts/7 is slave pty for gdb

Use <ctrl>C to terminate program
```



Going further with KGDB

- Good presentation from Doug Anderson with a lot of demos and explanations
  - Video: https://www.youtube.com/watch?v=HBOwoSyRmys
  - Slides: https://elinux.org/images/1/1b/ELC19_Serial_kdb_kgdb.pdf

